Front matter

Department of Electronic Systems

Examination paper for TFE4141 Design of Digital Systems 1
Date: December 15, 2025
Time (from-to): 15:00 – 19:00

SOLUTION EXAMPLE

Course contact: Per Gunnar Kjeldsberg
Present at the exam location: YES (approximately 16:30)

Permitted examination support material: C / Specified printed and hand-written support material is allowed. A specific basic calculator is allowed.
You are allowed to bring the course textbook (Dally, Harting and Aamodt, "Digital design using VHDL, a systems approach") as well as four (4) hand-written one-sided sheets of paper with your own notes.

OTHER INFORMATION

Get an overview of the question set before you start answering the questions.

Read the questions carefully and make your own assumptions. In your answers, explain clearly what assumptions you have made and how you have understood or limited the assignment
If there are direct errors or omissions in the assignment set and you cannot make your own assumptions, please refer to the information about complaints regarding formal errors on the NTNU website “Explanation of grades and appeals”.

Paper drawings: For questions **13** and **14** it is possible to submit the entire/some parts of the answer as handwritten sheets. Other questions must be answered directly in Inspera. At the bottom of the question you will find a seven-digit code. Fill in this code in the top left corner of the sheets you wish to submit.
Please note that only blue or black ballpoint pens are allowed
We recommend that you do this during the exam. If you require access to the codes after the examination time ends, click “Show submission”.
You are responsible for filling in the correct codes on the handwritten sheets. Therefore, read the cover sheet carefully. The Examination Office cannot guarantee that incorrectly completed sheets will be added to your assignment.

Weighting: The total number of achievable points is 100. The maximum number for each question is given on each question cover page.

Withdrawing from the exam: If you become ill or wish to submit a blank test/withdraw from the exam for another reason, go to the menu in the top right-hand corner and click “Submit blank”. This cannot be undone, even if the test is still open.

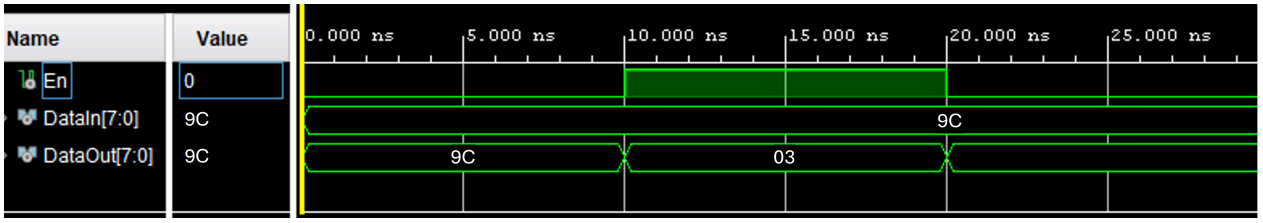
Access to your answers: After the exam, you can find your answers in the archive in Inspera. Be aware that it may take a working day until any hand-written material is available in the archive.

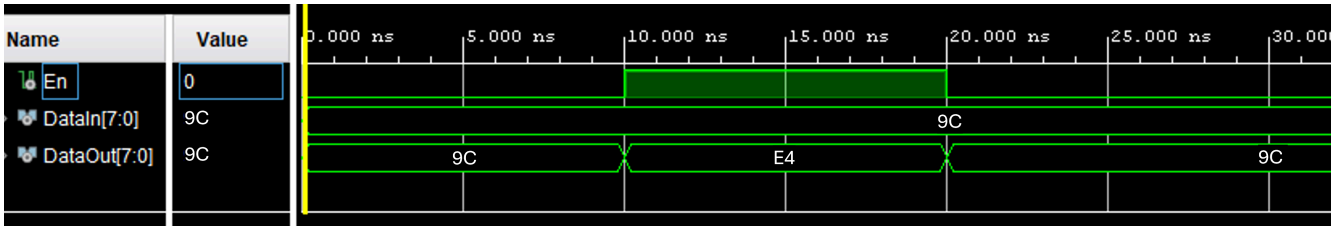
1 Simulation result (Correct answer 2 points, wrong answer -1 point, no answer 0 points)

Given the following VHDL code. Which alternative timing diagram reflects a correct simulation result?

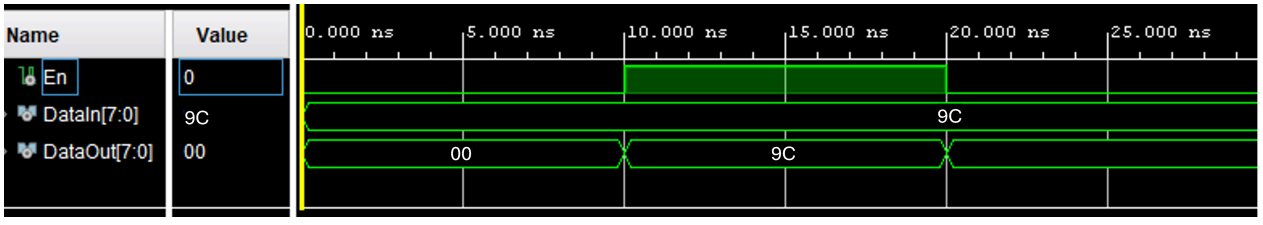
```
library IEEE;
use IEEE.NUMERIC_STD.ALL;

entity Task1 is
    Port ( En: in BIT;
           DataIn : in UNSIGNED (7 downto 0);
           DataOut : out UNSIGNED (7 downto 0));
end Task1;

architecture Behavioral of Task1 is
    constant Three: integer := 3;
begin
    DataOut <= (DataIn rol Three) when En = '1' else
                DataIn;
end Behavioral;
```

Select one alternative:





X



Maximum marks: 2

SOLUTION EXAMPLE: The hexadecimal DataIn value translated into binary is 9C = "1001 1100". After rotating left three bits the value is "1110 0100" = E4. When En = '1' the rorated value is placed on DataOut, while the original DataIn value is placed on DataOut when En = '0'. This is reflected in the alternative with an X above.

2 Resulting value (Correct answer 2 points, wrong answer -1 point, no answer 0 points)

Given the VHDL code below. Assume that we assign the values '1', '0', and 'Z' to insig1, insig2, and insig3, respectively. What is the resulting value on outsig?

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Task2 is
    Port ( insig1, insig2, insig3: in STD_ULOGIC;
           outsig : out STD_LOGIC);
end Task2;

architecture Behavioral of Task2 is
begin
    outsig <= insig1;
    outsig <= insig2;
    outsig <= insig3;
end Behavioral;
```

Select one alternative:

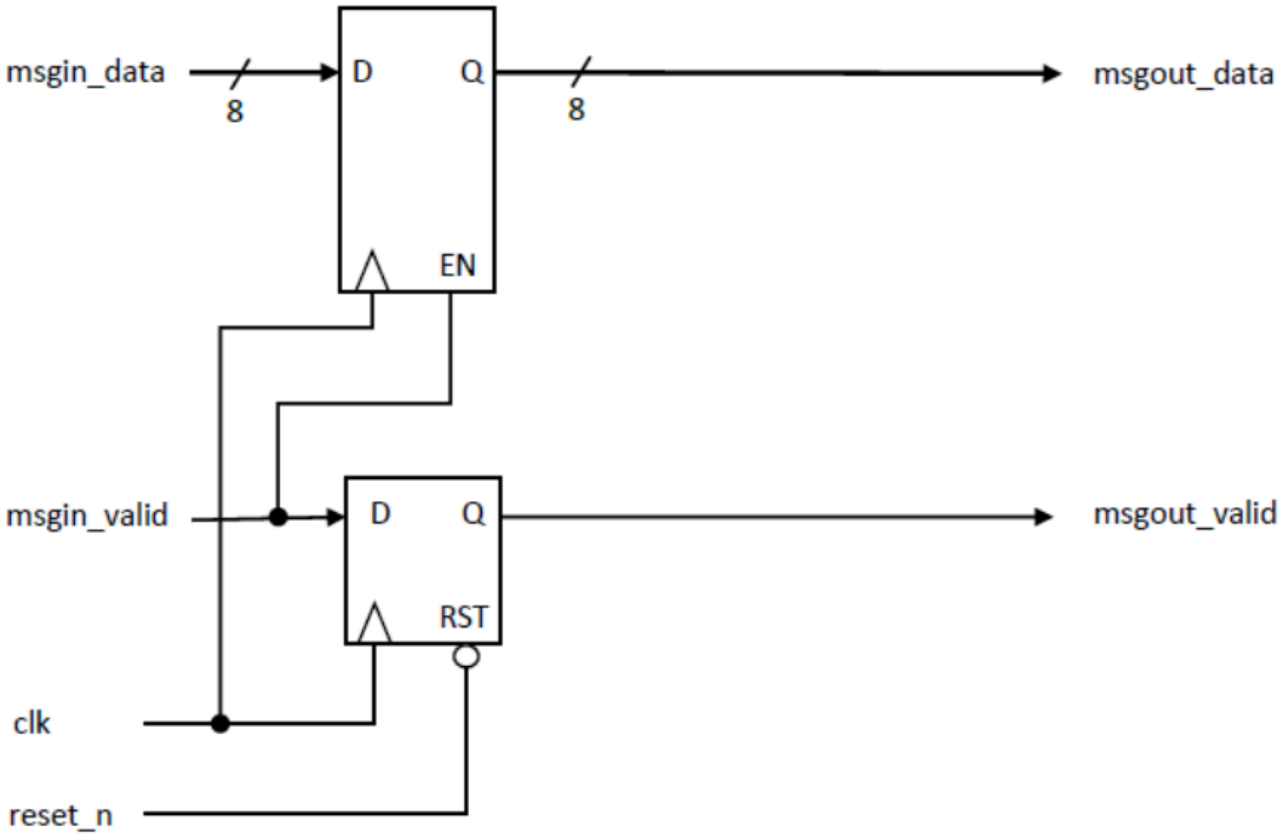
- 'Z'
- '1'
- ✗ 'X'

Maximum marks: 2

SOLUTION EXAMPLE: outsig is of type STD_LOGIC, which has built in resolution. The high impedance value 'Z' on insig3 does not influence the resulting value, while the competing '1' and '0' will result in an unknown value 'X'.

3 Matching representations (Correct answer 2 points, wrong answer -1 point, no answer 0 points)

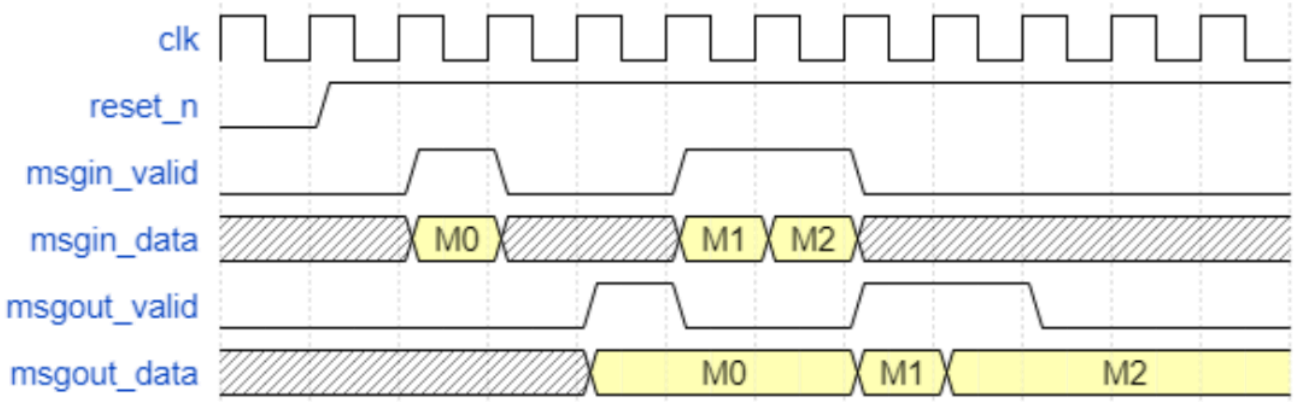
We have three representations of a digital circuit, a block diagram, a VHDL code, and a wave form. Which statement below reflects how the three representations match (i.e., they represent equivalent behaviours)?



SOLUTION EXAMPLE: The block diagram consists of two D-flipflops, one using msgin_valid as enable signal, and the other having an active low reset. This is reflected in the VHDL code, even though the block diagram does not explictely show that the enable is synchronized with the clock while the reset is asynchronous.

The wave form does not reflect the expected behavior of the two other representations. There is an added one clock period delay before the outputs change their value.

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity Task3 is
   port( signal clk : in std_logic;
         signal reset_n : in std_logic;
         signal msgin_valid : in std_logic;
         signal msgin_data : in std_logic_vector(7 downto 0);
         signal msgout_valid : out std_logic;
         signal msgout_data : out std_logic_vector(7 downto 0) );
end Task3;

architecture Behavioral of Task3 is
begin

   process(clk) is
   begin
      if(clk'event and clk = '1') then
         if(msgin_valid = '1') then
            msgout_data <= msgin_data;
         end if;
      end if;
   end process;

   process(clk, reset_n) is
   begin
      if(reset_n = '0') then
         msgout_valid <= '0';
      elsif(clk'event and clk = '1') then
         msgout_valid <= msgin_valid;
      end if;
   end process;

end Behavioral;
```



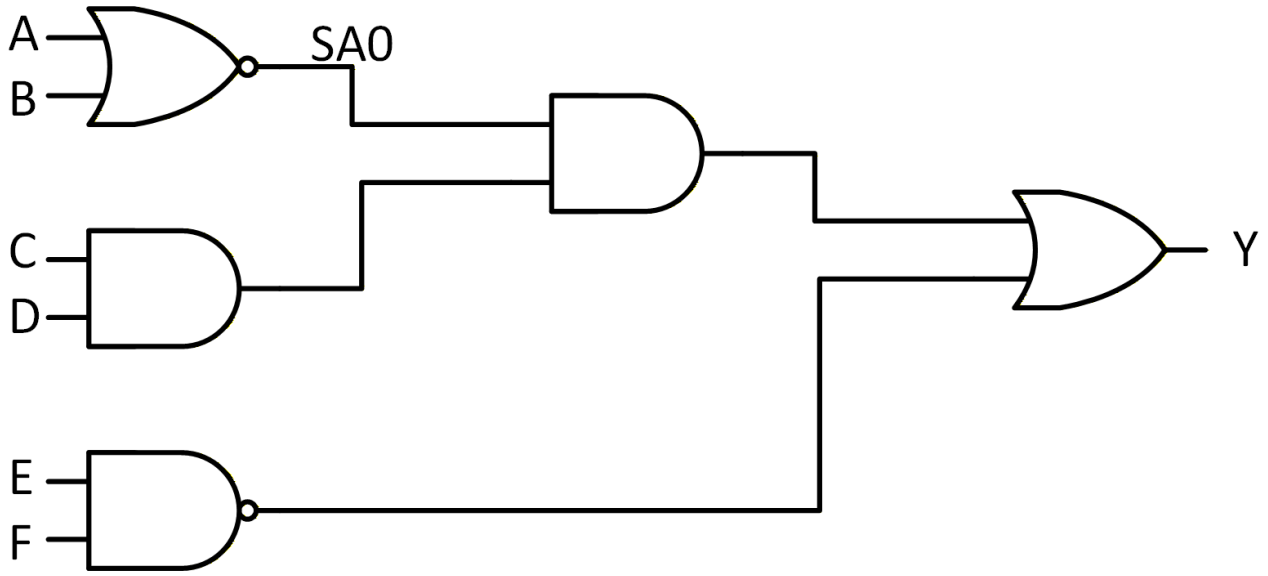
Select one alternative:

- The block diagram is matching both the VHDL code and the wave form
- The block diagram is only matching the VHDL code
- The block diagram is not matching any of the other representations

Maximum marks: 2

4 Test pattern (Correct answer 2 points, wrong answer -1 point, no answer 0 points)

Given the circuit below. Which statement is true related to a test for the output of the NOR-gate stuck at zero (SA0)?



Select one alternative:

- ABCDEF = 001111 is one of several possible test pattern
- There does not exist any test pattern for this stuck-at fault
- X The only possible test pattern is ABCDEF = 001111

Maximum marks: 2

SOLUTION EXAMPLE: In order to control the output of the NOR-gate to '1' (the opposite of the stuck-at value), both inputs A and B have to be '0'. For the resulting value at the output of the NOR-gate to propagate through the following AND-gate, the other input of the AND-gate has to be '1'. This requires that both input C and D of the leftmost AND-gate are '1'. Finally, to allow propagation through the OR-gate so that the result is observable on Y, the other input of the OR-gate must be '0'. This will only happen if both inputs E and F of the NAND gate are '1'. Hence, ABCDEF = 001111 is the only possible test pattern.

5 High level synthesis (Correct answer 2 points, wrong answer -1 point, no answer 0 points)

Given the following high-level code :

```
for ( i = 0; i <= 7; i++)
    Y = Y + A * indata[i] + B;
```

A and B are constant during one execution of the complete code but may change before a new execution. All input values (A, B, and each indata) are 8 bit vectors and can be accessed in parallel if needed.

Which statement is **correct** related to a digital implementation of the corresponding circuit?

Select one alternative:

- A fully combinatorial implementation requires 16 adders and 8 multipliers
- X If we unroll the loop twice (partial unroll = 2) we will need (at least) four clock cycles to complete the calculation
- Any micro architecture requires at least one multiplier and two adders.

SOLUTION EXAMPLE: See neext page

Maximum marks: 2

6 Testbench signals (Correct answer 2 points, wrong answer -1 point, no answer 0 points)

Given the following process written in a testbench. Which alternative timing diagram reflects a resulting wave form?

```
process
    constant period : time := 20 ns;
begin
    wait for (period * 0.7);
    stimuli <= '1';
    wait for (period * 0.3);
    stimuli <= '0';
end process;
```

Select one alternative:

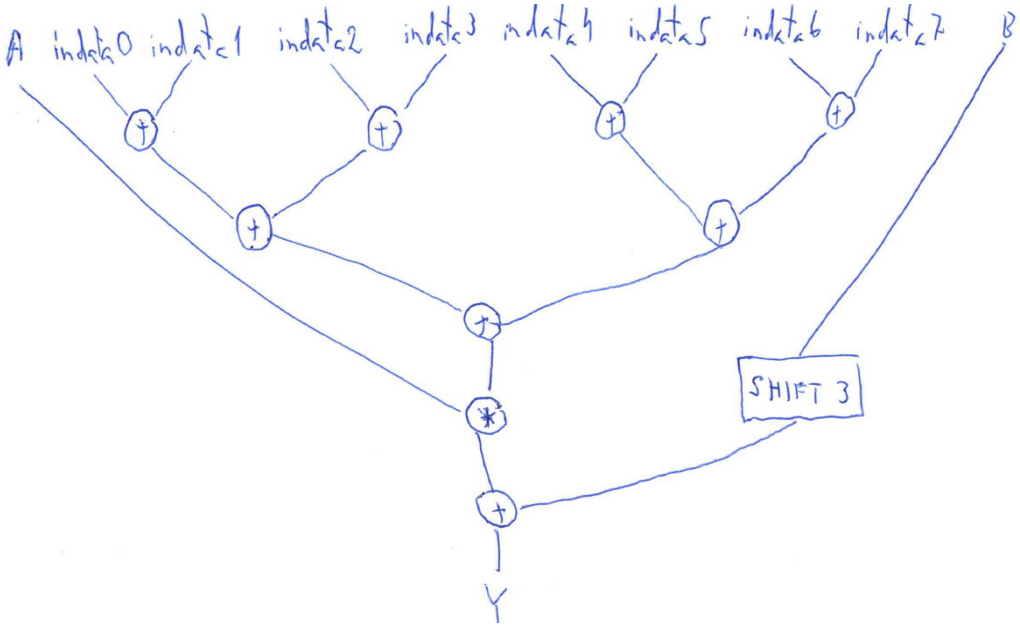
X

Maximum marks: 2

SOLUTION EXAMPLE: The process is a typical clock generator used in a testbench. It will repeat itself as long as the simulation runs. We can also see that it has a 70-30 duty cycle, as reflected in the wave form altetnative with an X above.

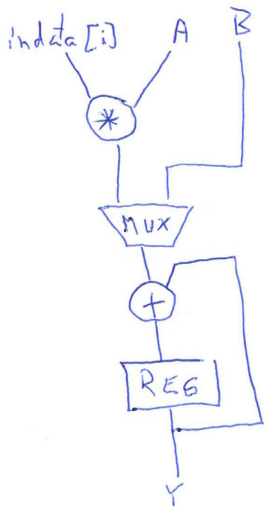
SOLUTION EXAMPLE 5:

The statement «A fully combinatorial implementation requires 16 adders and 8 multipliers" is wrong since there are several ways of reducing the number of multipliers and adders. Instead of multiplying A with each individual indata, we can first add the indata and then multiplying the result with A. This reduces the number of multipliers from eight to one. The addition of the indata requires seven adders. We also need to add B eight times. This can be done with eight adders, but since we are adding B * 8 we can simply shift B three bits to the left and use only one adder. A possible microarchitecture is shown below.



The statement "If we unroll the loop twice (partial unroll = 2) we will need (at least) four clock cycles to complete the calculation" is correct since we still need four iterations of the for-loop to complete the calculation. We need to save the intermediate Y value between each iteration and hence need at least four clock cycles.

The statement "Any micro architecture requires at least one multiplier and two adders" is wrong since the two add operations can share the same adder. A multiplexer is then needed to select between (A * indata[i]) and B as input to the adder. A possible microarchitecture is shown below.



7 What component (Correct answer 2 points, wrong answer -1 point, no answer 0 points)

What type of component does the VHDL code below implement?

```
use ieee.std_logic_1164.all;

entity Task7 is
   port( signal in1 : in std_logic;
         signal in2 : in std_logic;
         signal out1 : out std_logic);
end Task7;

architecture Behavioral of Task7 is
begin

   process(in1, in2) is
   begin
      if(in1 = '1') then
         out1 <= in2 after 2 ns;
      end if;
   end process;

end Behavioral;
```

SOLUTION EXAMPLE: The entity does not have a clock, so it cannot be a flipflop. Nor does it choose between two alternative inputs, so it cannot be a multiplexer. The signal in2 is connected to the output out1 when in1 is '1'. No information is given about the behavior when in1 is '0', and hence out1 should remain the unchanged, giving an "intentional" latch.

Select one alternative:

- Multiplexer
- Flipflop
- X Latch

Maximum marks: 2

8 Correct statement (Correct answer 2 points, wrong answer -1 point, no answer 0 points)

Which statement is correct?

Select one alternative:

- Due to its structured design, it is straight forward to perform a complete manufacturing test of a large multiplier.
- X A scan chain can reduce the number of pins needed for manufacturing testing.
- When writing a testbench for design verification you need to make sure that it is synthesizable.

Maximum marks: 2

SOLUTION EXAMPLE: It is NOT straight forward to perform a complete manufacturing test of a large multiplier. A scan chain improves both controllability and observability during manufacturing testing, and hence reduces the need for external pins for testing. Since a testbench is not a part of the final implemented chip, the testbench code does not have to be synthesizable.

9 True and false statements (Correct answer 2 points, wrong answer -1 point, no answer 0 points)

How many of the following two statements are **correct**?

- A generic constant can, for instance, be used to include different instants of the same module with different word widths in your design.
- A generate statement can, for instance, be used for simple structural repetition in your VHDL code.

Select one alternative:

- 1
- X 2
- 0

SOLUTION EXAMPLE: Both statements are true.

Maximum marks: 2

10 True or false (Correct answer 2 points, wrong answer -1 point, no answer 0 points)

How many of the following two statements are **correct**?

- Binding is a High Level Synthesis task where operations are assigned to operators. E.g., what multiplier should be used for a given multiplication in the code.
- Using as short abbreviations as possible for your signal names is in general a good idea in your VHDL code since the code then becomes more compact and easier to maintain.

Select one alternative:

- 2
- X 1
- 0

Maximum marks: 2

SOLUTION EXAMPLE: Binding does assign operations to operators in high level synthesis. Abbreviations should be used with care when you write your VHDL code since they tend to makes the code difficult to understand and maintain later (both for yourself and in particular for others).

11 Dynamic voltage and frequency scaling (Maximum 12 points)

You are designing a digital system where it is possible to dynamically scale the supply voltage to two alternative voltage levels and run the system at corresponding frequencies. The table below summarizes the two alternatives including the corresponding average energy consumption during one clock cycle.

Vdd [V]	2	1
Energy per cycle [nJ]	50	20
fmax [MHz]	50	10
cycle time [ns]	20	100

Assume now that the deadline for a certain task requiring 20 000 000 clock cycles is 1s. How would you adapt the voltage and frequency setting during execution of this task in order to minimize energy consumption?

Fill in your answer here

Format

Solution example:
To minimize energy consumption we should finish the task exactly at the 1 second deadline. We cannot solely
use the 1V setting since this only gives us half the number of clock cycles needed. If we use the 2V setting, we will finish much too early, vasting energy. Hence, we need to dynamically combine the two.

We can for example calculate the required period with the 2V setting as follows:

t x 50 000 000 + (1 - t) x 10 000 000 = 20 000 000
||
V
t = (20 000 000 - 10 000 000) / 40 000 000 = 0.25

This means that we first need to run our digital system at 2V for 0.25 seconds before we can change to 1V and run for the remaining 0.75 second.

Here we have ignored the switching time, but this can be a significant portion if the deadline is shorter.

Words: 0

Maximum marks: 12

The total energy conumption with this combination would be
$E = 0.25 \times 50\,000\,000 \times 50\times10^{-9} + 0.75 \times 10\,000\,000 \times 20\times10^{-9} = 0.775J$

The alternative, running at 2V all the time and then going to deep sleep (ignoring sleep energy) would give
$E = 20\,000\,000 \times 50\times10^{-9} = 1J$

12 Project phases (Maximum 8 points)

During digital design of a large system involving multiple teams of people, you typically have multiple phases in the project. Assume that your company starts with an *initial investigation*, followed by a *detailed investigation*, and finally *design & test*. Describe what activities typically takes place in each phase and, for *design & test*, what milestones you will expect to have.

Fill in your answer here

Format

Initial investigation (II):
- Small team. Architects with leads and managers
- Ideas are explored and evaluated
- Evaluation criterias:
  - Performance density increase (Performance per area)
  - Energy efficiecy
  - Area reduction
  - Performance increase
- Product requirements are determined
- The team decides roughly what the end product is going to look like

Detailed investigation (DI):
- More effort put into modelling of ideas
- Specification documents written and reviewed:
  - Architecture specification
  - Design specification for each block
  - Verification plan for each block
- Patents filed for inventions
- Product requirements broken down into engineering requirements and tasks
- Effort estimation and planning of tasks for design and test
- The team is increasing significantly in size

Design and Test (D&T):
- Writing of RTL code and testbenches is starting.
- Divided into three sub-phases by three milestones: Alpha, Beta and EAC.
- Alpha:
  - All blocks and testbenches implemented.
  - All interfaces (external and between blocks) are clearly defined.
  - Main functionality should work, but OK to have bugs.
- Beta:
  - Focus on improving performance, reducing area, improving energy efficiency, fixing bugs, improving pass rate, functional coverage, code coverage and other quality metrics.
  - Goals must be met for each metric.
- EAC:
  - Product available to Early Access Customers
  - All quality metrics fully met
  - No known bugs allowed
  - All reviews done

Words: 0

Maximum marks: 8

13 RSA cryptography (Maximum 20 points)

In the term project, we have created an RSA encryption circuit. This RSA encryption circuit is able to compute:

$$C = M^e \bmod n,\ M < n$$

a) (maximum 12 points)

Draw a block diagram for the datapath of your design. The block diagram should show all registers, mux'es, adders and shifters in the datapath of your design.

You can assume that any necessary precomputations have already been completed. You can omit the part of the datapath associated with serializing inputs and outputs. You can also omit everything related to control.

b) (maximum 8 points)

Use the block diagram from a) and explain how many clock cycles your design will need in order to encrypt/decrypt a 256 bit message using a 256 bit key.

Fill in your answer here

Format

SOLUTION EXAMPLE: See the three following pages

Words: 0

Maximum marks: 20

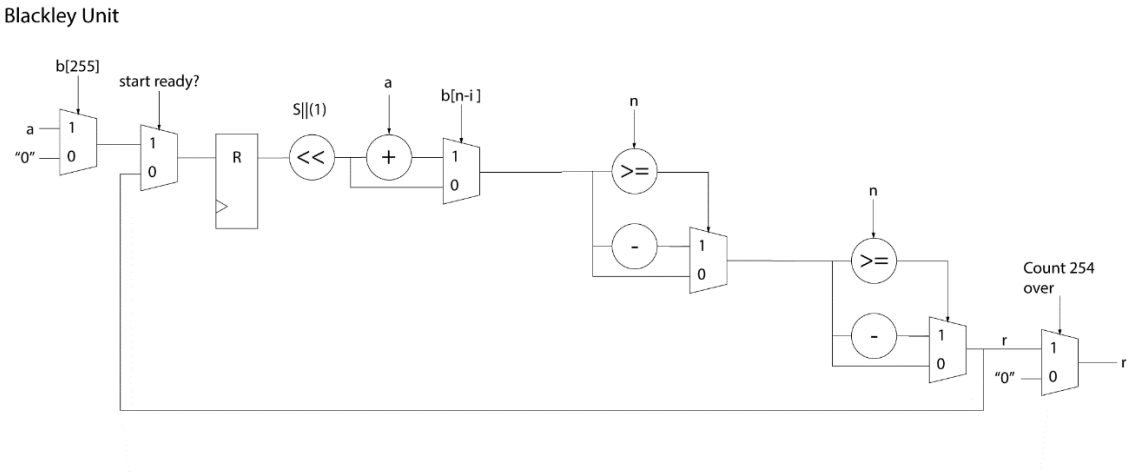
Solution examples Q13 :

Blackley

A)

Datapath in “Blakley”:

Generally, showing the Blakley RSA algorithm involves presenting a similar datapath, often referred to as a “Blakley Unit,” which performs Left-to-Right or Right-to-Left bitwise modular multiplication. This process consists of shifting the current result, adding the multiplicand if the current bit of the multiplier is ‘1,’ and then performing a conditional subtraction if the result exceeds N or a related threshold. These operations occur over the entire length of the multiplier, iterating through its bits, in this case, 256 bits.



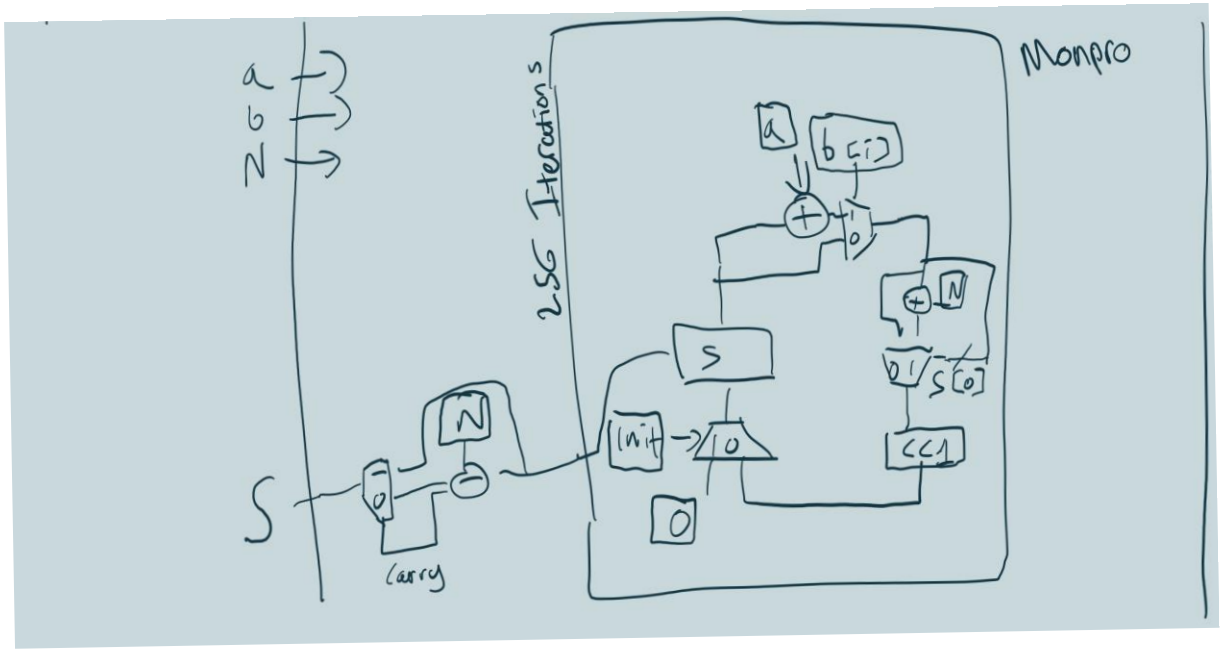
B)

In the Left-to-Right (LR) Square-and-Multiply method combined with Blakley’s modular multiplication, the operations are applied to an intermediate result that starts initialized to 1. For each bit of the exponent, this intermediate result is first squared (mod N), and if the bit is ‘1’, it is then multiplied by the base value (the original message). Both the square and the multiply are implemented using the Blakley Unit, which performs 256 iterations for a 256-bit operand. Therefore, in the worst case when every exponent bit is ‘1’ each bit triggers two modular multiplications (square + multiply), resulting in a maximum of 512 modular multiplication iterations per exponent bit. For a 256-bit exponent, this gives a worst-case execution time of 512 × 256 datapath cycles.

Montgomery:

a)

The Montgomery Method uses pre-computed variables to perform a domain transformation where the modulus R = 2^k replaces the original modulus N for intermediate steps. The primary datapath consists of the MonPro (Montgomery Product), which generates the product through a bit-by-bit iterative process. Initializing an intermediate accumulator S to zero, the datapath inspects the LSB of the multiplier in each clock cycle. If 1, the multiplicand is added to S. This is followed by a parity alignment step where, if S is odd, the modulus N is added to make the sum even. Afterwards there is a right-shift by 1 bit, effectively performing a step wise division by the radix. After k iterations, a final comparator and subtractor ensure the result is reduced below N, maintaining the value within the Montgomery domain for the next step of the square-and-multiply sequence.



b)

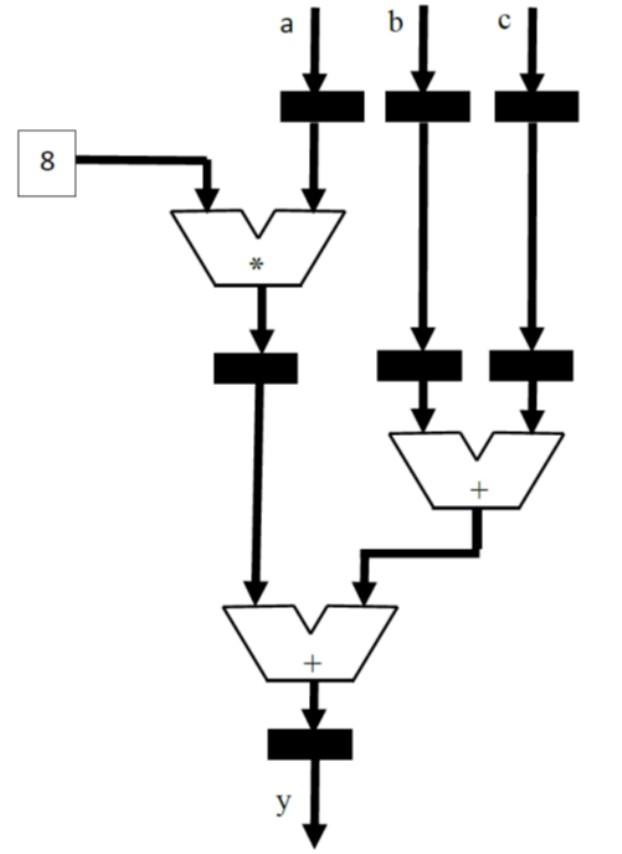
In the Montgomery-based square-multiply method, the process begins by shifting the base message and an initial value of 1 into the Montgomery domain through a pre-computed transformation. The architecture maintains an intermediate result, initialized to R mod N, which is updated as the controller iterates through each bit of the 256-bit exponent from left to right. For every bit, the datapath triggers a square operation by calling the MonPro unit with the intermediate result as both operands. If the exponent bit is ‘1’, it immediately triggers a second Multiply operation by calling MonPro again

with the squared result and the domain-transformed message. Because each MonPro call requires 256 internal iterations to complete, a worst-case scenario where every exponent bit is ‘1’ results in 512 MonPro calls (256 squares + 256 multiplies) plus the two initial and final domain transformations. This yields a total worst-case execution time of 514x256 MonPro iterations.

14 Design task (maximum 40 points)

Your boss has given you the block diagram below and asked you to be in charge of the design of the corresponding digital component. You are asked to improve both the area and the clock frequency of the design.

Inputs a, b and c are 16 bit wide signals of type unsigned. Black rectangles are registers of appropriate width. The square with the number "8" in it, indicates multiplication with the decimal number 8.



a) (max 10 points)

Explain how it is possible to modify the design above so that the clock frequency is doubled and the area is reduced. You should do so without adding any multiplexers. Draw a new block diagram reflecting the required changes.

b) (max 10 points)

Write synthesizable VHDL code that implements your design from a)

c) (max 10 points)

After having presented your design to your boss you are told that the inputs a, b, and c arrive sequentially on a common 16 bit wide bus. The system also includes a one bit signal START that indicates the arrival of input value a) and a one bit signal FINISHED which should indicate that the full calculation is finished.

You now need a state machine that controls the storing of data in registers and generates the FINISHED signal. Draw a state diagram and write synthesizable VHDL code for the state machine. Include a timing diagram that describes your assumptions regarding the relationship between the input and output signals.

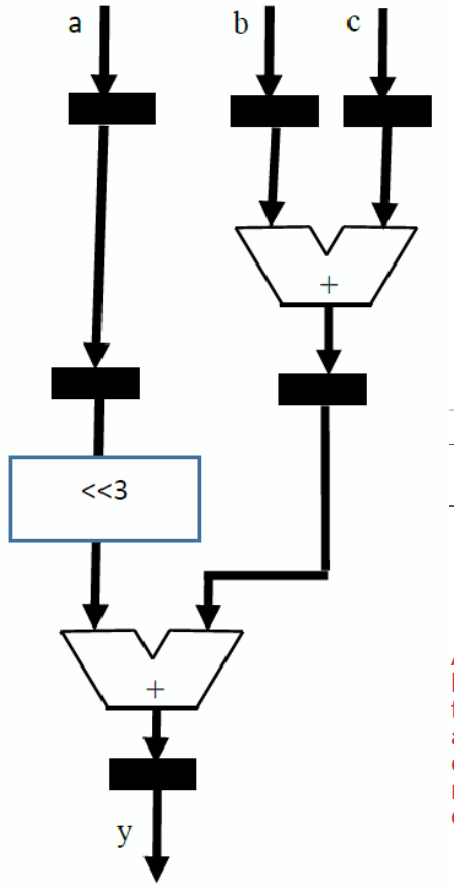
d) (max 10 points)

Write a testbench that verifies the functionality of the complete design, applying at least two sets of inputs (i.e., the START signal should be applied twice) and compares the calculated result with the expected result.

For maximum score, perform verification of your design from Task 14 c). If you have not completed Task 14 c) you can use the original design from the figure above, but then the max number of points you can achieve is 7.

Fill in your answer here

SOLUTION EXAMPLE:
a) Multiplication with 8 is the same as shifting three bits to the left. Hence, we do not need the multiplier. This reduces the area dramatically.
In addition, the upper adder can be moved one clock cycle earlier (between the two upper levels of registers). The clock frequency can then be doubled, since the critical path is now only through one adder instead of two. The resulting block digram is shown below.



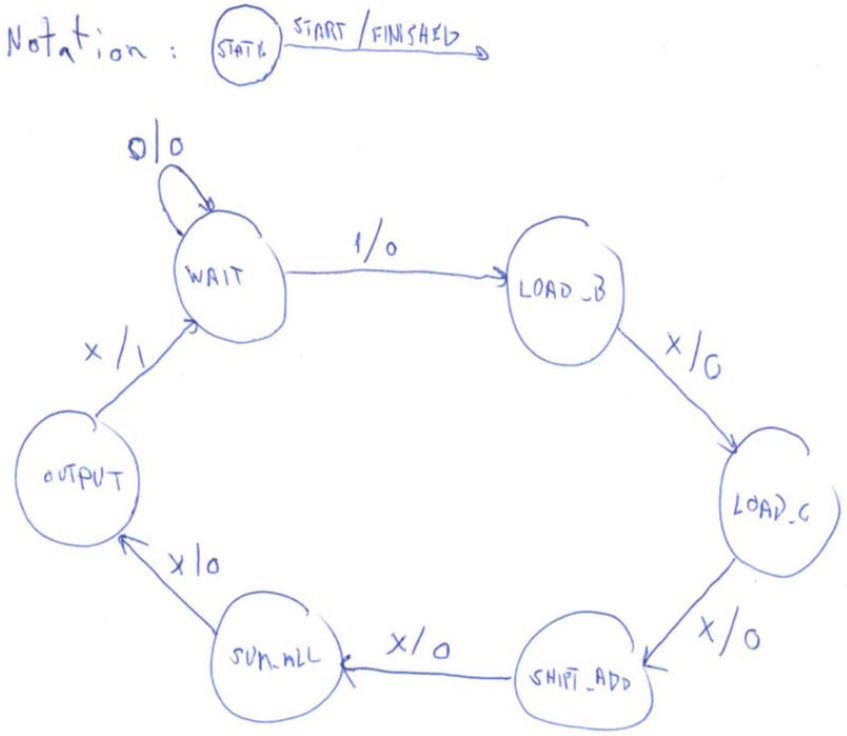
Words: 0

Maximum marks: 40

An additional possible optimization, which has not been explored further, is to exploit the fact that the tree LSB bits of the a-input is '0' after shifting. The last adder can hence avoid adding these bits and be made shorter. The three LSB of the result can be taken directly from the b+c addition.

b) See VHDL code for entity Shift_Add_Tree on next page.

c) A state diagram which handles the control is given below. In this case the storing av data and shift and add operations are implemented in the FSM itself as shown in the code Shift_Add_Tree_v2. Storing of the A data is performed in the WAIT state. Alternatively, the FSM could have generated enable signals for the registers implemented in Shift_Add_Tree from b).



SOLUTION EXAMPLE b) continued

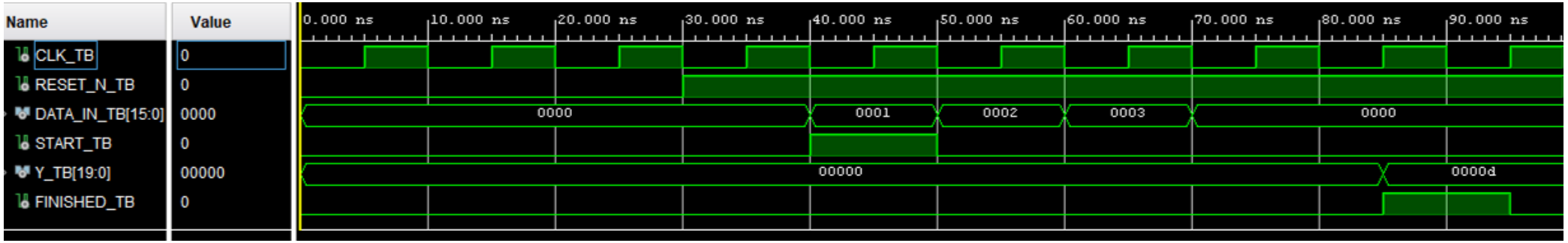
```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity Shift_Add_Tree is
    port (
        CLK       : in  std_logic;
        RESET_N   : in  std_logic;
        A         : in  unsigned(15 downto 0);
        B         : in  unsigned(15 downto 0);
        C         : in  unsigned(15 downto 0);
        Y         : out unsigned(19 downto 0)
    );
end entity Shift_Add_Tree;

architecture RTL of Shift_Add_Tree is

    signal Phase1_A : unsigned(15 downto 0) := (others => '0');
    signal Phase1_B : unsigned(15 downto 0) := (others => '0');
    signal Phase1_C : unsigned(15 downto 0) := (others => '0');

    signal Phase2_A_Shifted : unsigned(18 downto 0) := (others => '0');

    signal Phase2_Part_Sum  : unsigned(16 downto 0) := (others => '0');

    signal Phase3_Final_Y   : unsigned(19 downto 0) := (others => '0');

begin

    Y <= Phase3_Final_Y;

    Calc_Process : process(CLK, RESET_N)
    begin
        if RESET_N = '0' then
            Phase1_A <= (others => '0');
            Phase1_B <= (others => '0');
            Phase1_C <= (others => '0');
            Phase2_A_Shifted <= (others => '0');
            Phase2_Part_Sum  <= (others => '0');
            Phase3_Final_Y   <= (others => '0');

        elsif rising_edge(CLK) then

            Phase3_Final_Y <= ('0' & Phase2_A_Shifted) + ("000" & Phase2_Part_Sum);

            Phase2_A_Shifted <= Phase1_A & "000";

            Phase2_Part_Sum  <= ("0" & Phase1_B) + ("0" & Phase1_C);

            Phase1_A <= A;
            Phase1_B <= B;
            Phase1_C <= C;

        end if;
    end process Calc_Process;

end architecture RTL;
```

SOLUTION EXAMPLE c) continue

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity Shift_Add_Tree_V2 is
    port (
        CLK       : in  std_logic;
        RESET_N   : in  std_logic;
        DATA_IN   : in  unsigned(15 downto 0);
        START     : in  std_logic;

        Y         : out unsigned(19 downto 0);
        FINISHED  : out std_logic
    );
end entity Shift_Add_Tree_V2;

architecture RTL of Shift_Add_Tree_V2 is

    type state_type is (STATE_WAIT, STATE_LOAD_B, STATE_LOAD_C, STATE_SHIFT_ADD, STATE_SUM_ALL, STATE_OUTPUT);

    signal current_state : state_type;
    signal comb_state    : state_type;

    signal reg_A : unsigned(15 downto 0) := (others => '0');
    signal reg_B : unsigned(15 downto 0) := (others => '0');
    signal reg_C : unsigned(15 downto 0) := (others => '0');

    signal A_Shifted : unsigned(18 downto 0) := (others => '0');
    signal part_sum  : unsigned(16 downto 0) := (others => '0');
    signal final_Y   : unsigned(19 downto 0) := (others => '0');

    signal comb_A_Shifted : unsigned(18 downto 0);
    signal comb_part_sum  : unsigned(16 downto 0);
    signal comb_final_Y   : unsigned(19 downto 0);

begin

    Y <= final_Y;

    FSM_Mem : process(CLK, RESET_N)
    begin
        if RESET_N = '0' then
            current_state <= STATE_WAIT;

            reg_A <= (others => '0');
            reg_B <= (others => '0');
            reg_C <= (others => '0');
            A_Shifted <= (others => '0');
            part_sum <= (others => '0');
            final_Y <= (others => '0');

        elsif rising_edge(CLK) then

            current_state <= comb_state;

            A_Shifted <= comb_A_Shifted;
            part_sum  <= comb_part_sum;
            final_Y   <= comb_final_Y;

            case current_state is
                when STATE_WAIT =>
                    if START = '1' then
                        reg_A <= DATA_IN;
                    end if;
                when STATE_LOAD_B =>
                    reg_B <= DATA_IN;
                when STATE_LOAD_C =>
                    reg_C <= DATA_IN;
                when others =>

            end case;

        end if;
    end process FSM_Mem;
```

SOLUTION EXAMPLE c) continue

```
    FSM_Comb : process(current_state, START, reg_A, reg_B, reg_C, A_Shifted, part_sum, final_Y)
    begin
        comb_state       <= current_state;
        comb_A_Shifted   <= A_Shifted;
        comb_part_sum    <= part_sum;
        comb_final_Y     <= final_Y;
        FINISHED         <= '0';

        case current_state is

            when STATE_WAIT =>
                if START = '1' then
                    comb_state <= STATE_LOAD_B;
                else
                    comb_state <= STATE_WAIT;
                end if;

            when STATE_LOAD_B =>
                comb_state <= STATE_LOAD_C;

            when STATE_LOAD_C =>
                comb_state <= STATE_SHIFT_ADD;

            when STATE_SHIFT_ADD =>
                comb_A_Shifted <= reg_A & "000";

                comb_part_sum <= ("0" & reg_B) + ("0" & reg_C);
                comb_state <= STATE_SUM_ALL;

            when STATE_SUM_ALL =>
                comb_final_Y <= ("0" & A_Shifted) + ("000" & part_sum);
                comb_state <= STATE_OUTPUT;

            when STATE_OUTPUT =>
                FINISHED <= '1';
                comb_state <= STATE_WAIT;

        end case;
    end process FSM_Comb;

end architecture RTL;
```



SOLUTIONEXAMPLE c) continued

SOLUTION EXAMPLE d)

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use std.env.all;

entity Shift_Add_Tree_TB_V2 is
end entity Shift_Add_Tree_TB_V2;

architecture Behavioral of Shift_Add_Tree_TB_V2 is

    component Shift_Add_Tree_V2
        port (
            CLK       : in  std_logic;
            RESET_N   : in  std_logic;
            DATA_IN   : in  unsigned(15 downto 0);
            START     : in  std_logic;
            Y         : out unsigned(19 downto 0);
            FINISHED  : out std_logic
        );
    end component Shift_Add_Tree_V2;

    constant CLK_PERIOD : time := 10 ns;

    signal CLK_TB       : std_logic := '0';
    signal RESET_N_TB   : std_logic := '0';
    signal DATA_IN_TB   : unsigned(15 downto 0) := (others => '0');
    signal START_TB     : std_logic := '0';
    signal Y_TB         : unsigned(19 downto 0);
    signal FINISHED_TB  : std_logic;

begin

    UUT : Shift_Add_Tree_V2
        port map (
            CLK       => CLK_TB,
            RESET_N   => RESET_N_TB,
            DATA_IN   => DATA_IN_TB,
            START     => START_TB,
            Y         => Y_TB,
            FINISHED  => FINISHED_TB
        );

    CLK_GEN : process
    begin
        loop
            CLK_TB <= '0';
            wait for CLK_PERIOD / 2;
            CLK_TB <= '1';
            wait for CLK_PERIOD / 2;
        end loop;
    end process CLK_GEN;
```

```
    STIMULUS : process
    begin
        wait for 10 ns;

        RESET_N_TB <= '0';
        wait for CLK_PERIOD * 2;
        RESET_N_TB <= '1';
        wait for CLK_PERIOD;

        DATA_IN_TB <= to_unsigned(1, 16);
        START_TB   <= '1';
        wait for CLK_PERIOD;

        DATA_IN_TB <= to_unsigned(2, 16);
        START_TB   <= '0';
        wait for CLK_PERIOD;

        DATA_IN_TB <= to_unsigned(3, 16);
        wait for CLK_PERIOD;

        DATA_IN_TB <= (others => '0');
        wait for CLK_PERIOD;

        wait for CLK_PERIOD;

        assert FINISHED_TB = '1' report "Test 1 FAILED: FINISHED flag not set" severity error;
        assert Y_TB = to_unsigned(13, 20) report "Test 1 Failed" severity error;

        wait for CLK_PERIOD;

        DATA_IN_TB <= to_unsigned(5000, 16);
        START_TB   <= '1';
        wait for CLK_PERIOD;

        DATA_IN_TB <= to_unsigned(1000, 16);
        START_TB   <= '0';
        wait for CLK_PERIOD;

        DATA_IN_TB <= to_unsigned(2000, 16);
        wait for CLK_PERIOD;

        DATA_IN_TB <= (others => '0');
        wait for CLK_PERIOD;

        wait for CLK_PERIOD;

        assert FINISHED_TB = '1' report "Test 2 FAILED: FINISHED flag not set" severity error;
        assert Y_TB = to_unsigned(43000, 20) report "Test 2 Failed" severity error;

        wait for CLK_PERIOD * 2;

        report "Simulation Finished" severity note;
        stop;

    end process STIMULUS;

end architecture Behavioral;
```